Synchra: Private, Leveraged FX on Arc

Synchra Protocol Contributors

https://synchra.org

Abstract

Circle's Arc blockchain settles stablecoin value with sub-second finality and USDC-native gas, and Circle's own StableFX engine now settles *spot* FX conversion between stablecoins on Arc [2, 3]. Neither provides *leveraged* FX exposure, and neither hides who is trading what. Synchra is the layer above that: a clearing kernel that lets a trader post collateral once and open margined FX, crypto, and RWA positions against it, matched privately through a trusted execution environment (TEE) rather than a public order book. This paper describes the kernel, the portfolio margin model that makes leverage capital-efficient, the TEE matching design that makes it private, and where Synchra sits relative to existing onchain perpetuals (Ostium) and Arc's own settlement infrastructure (StableFX).

Keywords: portfolio margin, leverage, FX derivatives, TEE, private matching, stablecoin settlement

Contents

1 Introduction

Arc is a stablecoin-native chain: deterministic sub-second finality, USDC as native gas, EVM compatibility [2, 1]. In late 2025 Circle added StableFX, an RFQ engine that settles spot stablecoin-to-stablecoin FX conversion directly on Arc with payment-versus-payment finality [3]. Between Arc and StableFX, the settlement problem for stablecoin FX is solved.

What is missing is leverage and privacy. StableFX converts EURC to USDC at spot; it does not let a treasury hedge a EURC receivable with a margined FX forward, and every conversion it settles is visible onchain. A firm managing FX risk at size does not want its hedge ratio, timing, or currency exposure legible to anyone watching the chain.

Synchra is a clearing kernel and margin engine that sits above Arc's settlement layer. It does three things:

1. **Portfolio margin.** Collateral is posted once, against the whole account, not per position. A long EURC/USDC hedge can net against a USDC-denominated perpetual instead of each requiring its own margin.

2. **Private matching.** Orders are matched inside a TEE, not a public mempool or visible order book. The chain sees a settled trade; it does not see the order that produced it.

3. **Yield-bearing collateral.** Collateral sitting behind a position earns yield (USYC and similar instruments) instead of sitting idle.

FX is the first market because it is the one Arc's own roadmap is explicit about, and because Ostium already shows there is real demand for onchain leveraged FX/RWA exposure [8] – just not privately, and not with portfolio margin. Section 2 compares Synchra to both StableFX and Ostium directly.

1.1 What this paper covers

Section 2 places Synchra against StableFX and Ostium concretely. Section 3 defines actors and threat model. Section 4 gives the layered design. Section 5 is the clearing kernel; Section 6 is the portfolio margin formula and liquidation logic. Section 7 covers the market extension interface and the FX module specifically. Section 8 is the TEE matching design. Section 9 and 10 cover agent accounts and yield-bearing collateral. Section 11 is the fee model. Section 12 is a plain-language risk list. Section 13 is what ships first.

1.2 Risks, stated up front

- **Oracle dependency.** Margin and liquidation depend on price feeds; redundant oracles and circuit breakers reduce but do not eliminate this (Section 12).

- **TEE hardware trust.** Private matching assumes the enclave manufacturer (Intel/AMD) is honest. This is a real trust assumption, not a cryptographic guarantee.

- **Thin early liquidity.** FX pairs beyond EURC/USDC will have wide spreads until market makers build depth.

- **Smart contract risk.** Undiscovered bugs in the kernel could cause loss of funds despite audits and formal verification targets.

None of this is unique to Synchra – it is inherent to onchain leverage and private execution generally – but this paper is a design and a build plan, not an audited, production system.

2 Related Systems

Rather than a generic DeFi literature survey, this section names the three systems Synchra sits between.

	StableFX	**Ostium**	**Synchra**
Instrument	Spot FX conversion	Leveraged RWA/FX perps	Leveraged FX/RWA perps
Margin model	N/A (spot)	Pooled LP vault	Portfolio margin
Execution privacy	Public RFQ	Public	TEE-matched
Settlement chain	Arc	Arbitrum	Arc

Table 1: Synchra relative to the two systems closest to it.

StableFX [3] is Circle's own onchain FX engine: RFQ pricing aggregated across liquidity providers, payment-versus-payment settlement, live on Arc testnet since November 2025. It solves spot conversion. It has no leverage and no margin model, because it does not need one – a spot conversion settles and is done. Synchra does not compete with StableFX; a Synchra FX position can settle its spot leg through it.

Ostium [8] is a live perpetuals protocol (currently on Arbitrum) offering leveraged FX, commodities, and equity exposure against a single pooled LP vault, using a Pool-RFQ pricing model. It proves the demand for onchain leveraged FX exists – it carries real open interest today. It is fully transparent (every position, size, and PnL is public), and its risk model is per-position against a shared LP pool rather than portfolio margin, so LPs absorb concentrated counterparty risk rather than the protocol netting offsetting exposure.

Synchra's position: take the demand Ostium has already validated, remove the transparency, and replace the pooled-LP counterparty with portfolio-margined, kernel-settled positions – on the chain whose settlement layer (StableFX, USDC/EURC, CCTP) is purpose-built for exactly this instrument class.

3 System Model

3.1 Actors

Traders open, adjust, and close positions through the execution layer.

Agents autonomous accounts with a cryptographic identity and a scoped, signed permission grant – not scripts a trader runs, but accounts the kernel recognizes directly (Section 9).

Market makers quote into the RFQ / private matching flow and are compensated through spread and funding.

Liquidators close underwater accounts and earn a penalty share.

3.2 Threat model

- Consensus security is inherited from Arc; we assume Byzantine fault tolerance at that layer.
- Smart contract risk is addressed through formal verification of the kernel and independent audits of each market extension.

- Oracle manipulation is mitigated with redundant feeds, TWAP comparison, and circuit breakers (Section 12).
- TEE compromise is treated as low-probability but real: it would leak order flow, not let an attacker forge a settlement, and the kernel does not depend on the TEE for correctness, only for confidentiality.
- Metadata leakage (timing, request frequency) is out of scope for this design.

3.3 Design goals

G1 Collateral posted once must be usable across every open position, subject to portfolio risk limits.

G2 Margin is computed on the whole account, not per position.

G3 A new market type plugs into the kernel through one interface, without kernel changes.

G4 Order details (price, size, side) are not visible before settlement.

G5 Collateral earns yield while backing positions.

G6 Agents are accounts the kernel recognizes directly, with their own permission scope.

4 Architecture

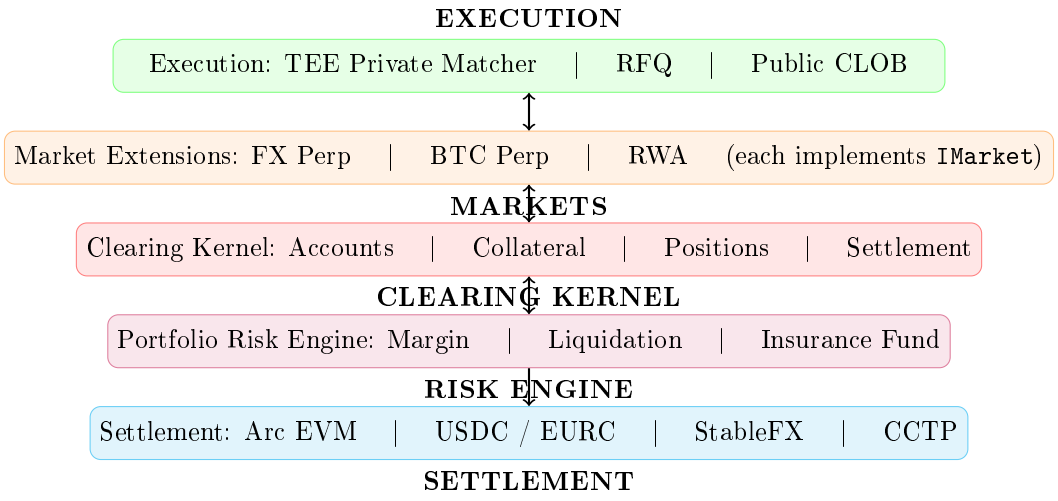

Figure 1: Synchra layers. Only settlement is one-directional – everything above it can query and be queried.

Three rules govern the layering: (1) a market extension cannot touch kernel state except through the interface in Section 7; (2) the kernel implements only accounting, collateral, and settlement – nothing market-specific; (3) execution (matching) can be off-chain or TEE-based, but settlement is always a deterministic on-chain state transition.

5 Clearing Kernel

5.1 Account and collateral

An account is a tuple $A = (\mathcal{B}, \mathcal{P}, \mathcal{H})$: collateral balances $\mathcal{B}$ by asset, an open position set $\mathcal{P}$ by market, and a credit/debit history $\mathcal{H}$.

Tier	Assets	Haircut
1	USDC, EURC	0%
2	USYC, stUSD	2%–5%
3	ETH, BTC (liquid staking)	10%–20%
4	Tokenized RWAs	15%–35%

Table 2: Collateral tiers. A cross-chain (CCTP) asset keeps its tier and adds a +2%–5% haircut for bridging latency.

Effective collateral: $\mathcal{C}_{\text{eff}} = \sum_a b_a \cdot (1 - h_a) \cdot p_a$, summed over balance b_a, haircut h_a, and oracle price p_a.

5.2 Trade lifecycle

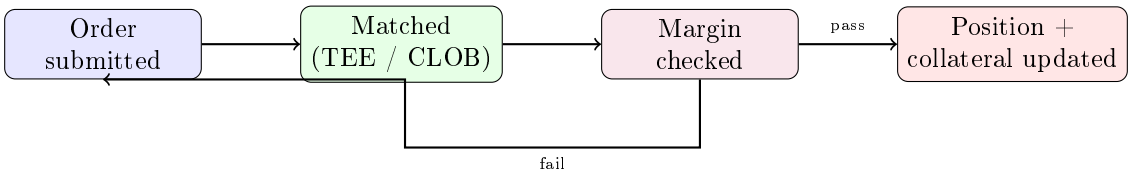

Figure 2: Every trade takes this path regardless of which market extension or execution mode produced the match.

Funding settlement and liquidation settlement follow the same shape: an event triggers a kernel-computed balance change, applied atomically with the position update. Positions are stored as opaque bytes with typed accessors – the kernel does not interpret market-specific fields, it only enforces that collateral changes are atomic with position changes.

6 Portfolio Margin

Margining each position independently over-collateralizes hedged accounts. A trader long 10 BTC-notional on one market and short 9.5 BTC-notional on another has 0.5 BTC of real risk but would post margin on 19.5 BTC of gross notional under isolated margin. Synchra margins the account, not the position.

$$\mathcal{M}(\mathcal{P}) = f_S(\mathcal{P}) + f_C(\mathcal{P}) + f_L(\mathcal{P}) + f_K(\mathcal{P})$$

- f_S – **scenario margin**: worst-case loss across a scenario set $\mathcal{S}$ (parallel price shifts, funding-rate spikes, correlation breakdown): $f_S(\mathcal{P}) = \max_{s \in \mathcal{S}} \sum_{p \in \mathcal{P}} \big(V_p(s) - V_p(\text{current})\big)$.
- f_C – **concentration charge**: penalizes exposure concentrated in one asset group beyond a threshold θ.
- f_L – **liquidity charge**: $\sum_p |\text{size}_p| \cdot \ell_p$, larger for thinner markets.

- f_K – **correlation adjustment**: for signed sizes s_i, s_j and correlation ρ_{ij}, $f_K = \beta \sum_{i \neq j} \rho_{ij} \cdot s_i \cdot s_j$. Opposite-direction correlated positions ($s_i s_j < 0$) reduce margin – a genuine hedge; same-direction correlated positions ($s_i s_j > 0$) increase it, since that is concentrated risk, not diversification. Sign matters: using unsigned sizes here would grant a margin discount to same-direction bets, which is wrong.

MVP ships two markets (Section 13), but EURC/USDC and BTC/USDC are not economically correlated, so f_C/f_K still have nothing meaningful to act on between them. f_S and f_L do real work from day one; f_C/f_K become load-bearing once a second correlated market exists – a second FX pair, or two accounts taking opposite sides of the same pair.

6.1 Hedging is not a new feature

A trader is "hedged," in this model, whenever they hold two correlated positions in opposite directions – f_K prices that automatically. Nothing extra needs to be built for a trader to hedge manually: open the offsetting position, watch $\mathcal{M}(\mathcal{P})$ drop. What *is* new infrastructure is a strategy vault (Section 10) automating that same offset on a rule set instead of a human clicking twice – a sub-account with its own rebalance trigger, trading through the same kernel path as any other account. The margin math is identical either way; a vault only adds the automation layer on top of it.

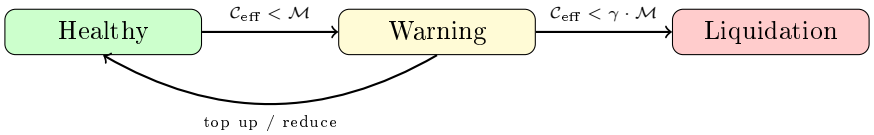

Figure 3: Liquidation state machine. $\gamma \approx 0.8$–0.9. Liquidation itself is tranched, not all-at-once, to avoid cascades (Section 12).

The insurance fund target scales with the square root of aggregate squared position size, $F^* = \kappa \sqrt{\sum_p \text{size}_p^2}$, with κ calibrated against historical stress periods before mainnet.

7 Market Extensions

A market extension implements one interface and a handful of lifecycle callbacks; the kernel never needs to know what the market *is*.

Figure 4: `IMarket` (simplified)

```
struct MarketPosition { marketId, size (signed), entryPrice,
margin, leverageCeiling }
getPnL(oraclePrice)   getFunding(position, period)
validateOpen(size, collateral)   validateClose(position)
onOpen / onClose / onLiquidate(user, market, position)
```

`leverageCeiling` is a per-market bound, not a trader's effective leverage – because margin is computed at the account level (Section 6), effective leverage is total notional over $\mathcal{C}_{\text{eff}}$, an account-level number the risk engine computes, not something read off one position.

7.1 FX perpetual (first module)

EURC/USDC at launch, additional pairs after. Funding follows the interest-rate differential, $r_{\text{FX}} = r_{\text{quote}} - r_{\text{base}}$. The spot leg of a settled trade can route through StableFX's PvP settlement rather than requiring Synchra to bootstrap its own EURC/USDC liquidity. This is the module the rest of the paper's examples assume.

7.2 Later modules (not required for the FX thesis, kept for continuity)

A BTC/USDC perpetual proves the interface generalizes beyond FX. RWA and rate modules (tokenized Treasuries, USYC yield trading) and strategy vaults (delta-neutral, FX carry, funding arbitrage) follow the same interface once a second correlated market makes portfolio margin's f_C/f_K terms meaningful. None of these require kernel changes to add.

8 Private Matching

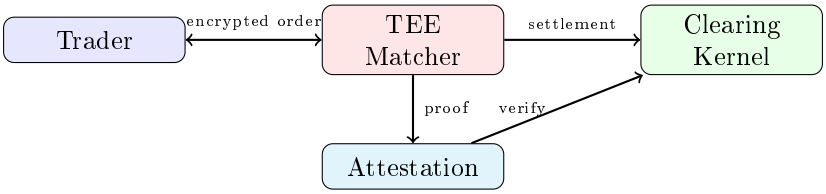

Figure 5: An order is only ever decrypted inside the enclave. The chain sees an attested settlement, never the order.

The trader encrypts an order to the TEE's public key; the enclave decrypts, matches, and produces a signed attestation that on-chain verification checks before the settlement is accepted. If attestation fails, the system falls back to public settlement rather than blocking the trade. Verifying a TDX/SEV-SNP attestation directly on-chain costs roughly 75M gas; compressing it with a ZK proof of the attestation (following Kalypso's approach [7]) brings that to roughly 300K gas – a defense-in-depth layer over the TEE, not a replacement for it. Side-channel attacks on the enclave (cache timing, power analysis) are acknowledged and out of scope for the current design.

8.1 On-chain footprint of a private position

Matching an order privately is only half the claim – the position that results has to stay private too, or the TEE only hid the five seconds before a trade. Synchra keeps two things off-chain in plaintext:

- **Sealed position parameters.** Side, leverage, entry price, size, and any TP/SL thresholds are never stored as separate on-chain fields. They are collapsed into one `sealedParams: bytes` blob, encrypted to the TEE's key, alongside a plain `collateral` amount and `status` the kernel needs for solvency accounting. Any check that would otherwise read a plaintext field – a TP/SL trigger, a liquidation solvency check, a margin recomputation – runs *inside* the enclave against the decrypted blob; the contract never re-derives it from public state.
- **Portfolio linkage.** Cross-margined positions are grouped by a `portfolioKey` – a hash the TEE derives from the account's private key material, not the account address itself.

The risk engine still sees the full position set behind a `portfolioKey` when it computes $\mathcal{M}(\mathcal{P})$ (Section 6), because it has to; an outside observer sees only that some set of positions shares a key, not whose account that is or what the set contains.

The kernel authorizes exactly one caller – the attested TEE – to invoke settlement, liquidation, and TP/SL-trigger functions. It does not recompute PnL or check thresholds against plaintext; it checks that the caller is the attested TEE and that its own invariants hold (collateral bounds, no double-settlement), then applies the number the TEE provides. This means events can be stripped to the minimum needed for indexing:

Event	**Emits**
Position opened	`positionId`, `collateral`
Position closed / liquidated	`positionId` only
Trade settled	`matchId` only

Table 3: No event carries price, size, side, or PnL. Oracle price and settlement amount are consumed by the kernel from the TEE's authenticated call, not re-emitted.

This is deliberately not full position-level ZK privacy (commitment/nullifier circuits proving position validity without any party but the trader ever seeing the plaintext, as in comparable Stellar-based designs). That is a stronger, ZK-native version of the same idea, and a heavier build; sealed params plus an authorized-settler contract gets most of the same on-chain opacity with Solidity and a TEE, no circuit design required, and composes cleanly with the ZK layer already on Synchra's roadmap (Section 13) if solvency-proof-style privacy is needed later.

Two other execution modes exist for markets that do not need this: a **public CLOB** for transparent price discovery, and **RFQ** for block trades where a taker requests quotes and settles atomically against the best one.

8.2 Why this is not an on-chain order book

The TEE matcher keeps full order-book semantics – continuous matching, price-time priority, standing orders – but the book itself lives in enclave memory, not contract storage; the chain only sees attested settlements. A literal on-chain order book with privacy (order state encrypted in contract storage, matched by FHE or a ZK circuit without ever decrypting) is a harder, still-immature problem: continuous FHE compute per match is expensive today, and putting per-order state on-chain works directly against Section 6's cost profile, where the expensive operation is already the full-portfolio margin recomputation, not order placement. TEE matching gets the useful property – a private, continuously-matched book – without inheriting that cost. A ZK-proved batch auction (commitments on-chain, a proof that the clearing price was computed correctly, no plaintext order ever posted) is the more realistic next privacy-preserving execution mode, and is already covered by the batch-verification proof in Section 12's ZK layer – it is a variant of an existing roadmap item, not a new one.

9 Agents

An agent account is a normal clearing account with a cryptographic identity and a scoped, signed capability instead of a human signer behind every action:

$$\text{capability} = \text{Sign}_{sk_{\text{owner}}}\big(H(\text{agent\_id}, \text{action}, \text{limits}, \text{expiry})\big)$$

The kernel checks the owner's signature before honoring any action under the capability. Execution policy (max position size, max leverage, daily loss limit, allowed markets) is enforced at the kernel level, not trusted to the agent's own code. Session keys are ephemeral and scoped, so an agent never needs the owner's master key. High-frequency agent-to-agent transfers (data fees, per-call payments) use Circle's Nanopayments so sub-cent transfers stay economical [5], and agent identity is designed to be compatible with Circle's Agent Stack [4].

10 Yield

Collateral tiers (Table 2) are not just haircut buckets – each accrues yield to the depositor while still counting toward margin. USDC/EURC can sweep into overnight repo; USYC carries its own money-market yield [6]; RWA collateral earns its underlying's yield at a wider haircut. Yield accrual never reduces an asset's margin eligibility – value for margin purposes is market value plus accrued yield.

Strategy vaults are the same mechanism at the account level: a vault is a sub-account with its own risk isolation, trading through the same kernel and margined the same way as any trader, so a vault-specific strategy failure cannot leak into other accounts. This is also how CLOB/RFQ liquidity gets provided – market-making is one more vault strategy, not a separate pooled-liquidity system with its own risk model.

11 Economics

No protocol token. Every incentive is paid in stablecoins or yield-bearing assets, which keeps regulatory treatment simple and avoids emissions-driven liquidity that has to be unwound later.

Fee	**Rate**	**Goes to**
Taker	0.03%–0.06%	LPs, treasury
Maker	0%–0.02%	LPs
Vault performance	10%–20%	Vault manager, treasury
Yield keep	0%–10 bps	Treasury
Liquidation penalty	1%–5%	Liquidator, insurance fund

Table 4: Fee schedule. Dynamic within these ranges by volume tier.

The tradeoff of no token: early liquidity has to be earned with real fee yield and USYC-denominated incentives from the treasury rather than bootstrapped with emissions, which is slower but does not need to be unwound later.

12 Security

Oracles Pyth (primary), Chainlink (staleness check), CLOB TWAP (circuit-breaker reference). A δ divergence (1%–5%) between primary and secondary pauses the affected market.

Liquidation cascades Positions are liquidated in tranches, ordered by time-in-distress, with a volume-based circuit breaker if total liquidation flow in a window exceeds a threshold.

Kernel invariants Collateral cannot be created or destroyed outside authorized operations; position and collateral updates are atomic; settlement is deterministic given the same input state; a market extension cannot touch kernel state outside its interface.

TEE Compromise leaks order flow, not funds – settlement correctness never depends on the enclave, only confidentiality does.

13 Roadmap

13.1 MVP: private, portfolio-margined EURC/USDC (and BTC/USDC) perpetuals

Every component below is real and working end-to-end, not a stub – the point of the MVP is proving all three pillars together, not demoing them separately.

- Clearing kernel (Solidity, Arc EVM, Foundry): accounts, collateral, positions, settlement.
- One FX market extension (EURC/USDC) plus the existing BTC/USDC extension, both against the same `IMarket` interface.
- Full portfolio margin engine ($f_S + f_C + f_L + f_K$), on-chain enforced, off-chain computed.
- TEE-matched private RFQ against a real enclave (Intel TDX via Phala or Marlin), with on-chain attestation verification and public-settlement fallback.
- Agent accounts: identity, capability tokens, session keys, one working agent type (trading agent), Nanopayments-integrated.
- USYC as a live collateral tier.
- Frontend (Next.js) via Circle Developer Controlled Wallets; Arc Testnet deployment.

13.2 Phase 1 – hardening

Formal verification of kernel and margin invariants; wider scenario coverage in f_S (vol shocks, correlation-breakdown regimes); additional crypto markets; multi-asset collateral with dynamic haircuts; mainnet.

13.3 Phase 2 – more markets

Additional FX pairs (GBP/USD, JPY/USD); RWA and rate module; strategy vaults; internal repo market; CCTP v2 cross-chain collateral.

13.4 Phase 3 – scale

ZK solvency attestation for institutions; ZK-compressed TEE attestation verification; institutional sub-accounts; agent strategy vaults (smart treasuries); migration path to Arc's protocol-level privacy roadmap (APS) if and when it ships.

14 Conclusion

Arc settles stablecoin value. StableFX settles spot FX conversion on top of it. Neither gives a trader leverage, and neither hides an order before it executes. Synchra is the layer that does both: post collateral once, get portfolio-margined leverage across correlated positions, match privately through a TEE, settle through the same rails Circle already built. Ostium already proved leveraged onchain FX/RWA has demand; Synchra's bet is that the next version of that market is private and portfolio-margined, on the chain built specifically for stablecoin-native FX.

Acknowledgements. We thank the Circle Arc and Encode Club communities for the accelerator context that motivated this work.

References

[1] Circle Internet Financial. Arc technical architecture: Malachite consensus and reth execution. https://circle.com/arc/technical, 2025.

[2] Circle Internet Financial. Circle arc: Programmable settlement for financial applications. https://circle.com/arc, 2025.

[3] Circle Internet Financial. Circle stablefx: An onchain fx engine for arc. https://www.circle.com/blog/introducing-circle-stablefx-and-circle-partner-stablecoins, 2025.

[4] Circle Internet Financial. Agent stack: Building autonomous agents on arc. https://circle.com/agent-stack, 2026.

[5] Circle Internet Financial. Nanopayments: Gas-free sub-cent usdc transfers powered by circle gateway. https://circle.com/nanopayments, 2026.

[6] Circle Internet Financial. Usyc: Tokenized money market fund. https://circle.com/usyc, 2026.

[7] Marlin Protocol. Kalypso: Tee attestation verification via zk proofs on symbiotic. https://marlin.org/kalypso, 2026.

[8] Ostium Labs. Ostium: Perpetual swaps for real-world assets. https://www.ostium.com, 2026.